

PDF 5 - Testing, Reliability & Submission Pack

Tests, edge cases, README/Research/Design/AI_USAGE templates and the final reviewer demo checklist

CipherSchools Hiring Assignment - LLD Practice Platform | September 2026

1. Test strategy

Gemini-specific reliability tests: mock Gemini at the adapter boundary; test timeout, provider error, malformed JSON, missing rubric criteria, out-of-range scores/confidence, and successful normalized feedback.

Unit/integration tests should never require a real Gemini API key.

Level	Must test
Domain unit	Attempt cannot autosave after submit; Evaluation legal transitions; retry only from FAILED; weighted score calculation
Validation unit	Missing assumptions/classes; oversized payload; malformed AI output; out-of-range scores
Service integration	Submit transaction creates snapshot + evaluation exactly once; ownership checks; idempotency
Evaluator contract	Provider response -> domain result; prompt-injection text treated as data; parse failure
E2E	Problem -> start -> fill -> submit -> completed feedback -> history
Failure E2E/API	Provider failure shows Failed and retry; double-click submit creates one submission

2. High-value edge cases

- Empty or whitespace-only fields.
- Learner lists classes but gives no responsibilities.
- Duplicate class names.
- Submit invoked twice from two browser tabs.
- Evaluation job receives the same event twice.
- AI returns missing criterion, score 9/5, invalid confidence, or Markdown instead of JSON.
- User tries another user attempt URL.
- Attempt history has one attempt only; compare screen should not break.
- Evaluator fails after deterministic checks but before storing AI result.

3. README structure

```
# LLD Practice Platform
## Problem statement
## MVP and learner journey
## Tech stack
```

Prepared as an implementation blueprint for the 2-day engineering assignment

```

## Architecture (modular monolith)
## Domain model and important interfaces
## Evaluation approach (deterministic vs AI)
## Local setup
## Environment variables
## Seed/demo credentials
## Running tests
## Key trade-offs
## Failure handling
## Limitations
## What I would build next
## Deployment URL

```

4. DESIGN.md structure

- Why structured text was selected for the MVP and what evidence it provides.
- Core entities and why each responsibility lives there.
- Attempt and Evaluation state diagrams.
- Evaluator and submission-format variation points.
- Rubric dimensions and why AI is not used for deterministic checks.
- How Change Test A and B are handled without rewriting the practice flow.
- Why the architecture is a modular monolith and what component would separate first at scale.

5. RESEARCH.md / 1-2 page note

Section	Content
Learner problem	LLD is open-ended; learners lack explainable feedback and confidence in trade-offs
Tools reviewed	Educative, DesignGurus, LLDCanvas, LLD Arena or similar
What works	Problem libraries, repeatable frameworks, UML/code examples, staged hints, rubrics/tests
Gap	Many resources teach/show solutions; fewer center a short attempt -> explainable feedback -> retry loop
Product direction	Small problem set + structured submission + evidence-backed rubric + history/compare

6. AI_USAGE.md template (required by assignment)

Document the provider choice explicitly: "Selected Google Gemini via Google AI Studio for the MVP and rejected adding OpenAI because a second provider adds complexity without improving the learner loop. Preserved an Evaluator interface so providers/evaluation approaches can be swapped later."

```

# AI Usage
1. Domain model
AI suggested: ...
Accepted: ... because ...
Rejected: ... because it added unnecessary abstraction.

```

```

2. Evaluation rubric
AI suggested: ...

```

Changed: ... to require evidence and fixed criteria.

3. Frontend state/performance

AI suggested: ...

Accepted/rejected: ...

4. Security/reliability review

AI found: ...

Action taken: ...

5. Testing

AI proposed edge cases: ...

Added: ... / skipped: ... and why.

7. Final reviewer demo script (4-6 minutes)

1. Open problem library; explain narrow MVP and why you chose structured text.
2. Open Parking Lot; show requirements and rubric preview.
3. Start attempt; fill/preview classes and trade-offs; point out autosave.
4. Submit; show immutable snapshot and evaluation state.
5. Open feedback; show evidence-backed criterion card, not just a score.
6. Open history; compare a previous attempt and demonstrate learning loop.
7. Briefly open domain interfaces/state machine and explain change tests.
8. Show tests + AI_USAGE.md + one failure case (evaluator fails/retry).

8. Submission checklist

- Repository is public/accessible as required; no .env committed.
- Live demo URL works in incognito.
- Fresh clone setup tested.
- README, RESEARCH.md, DESIGN.md, AI_USAGE.md included.
- At least important unit tests + few failure/edge cases pass.
- No console errors, broken loading states, dead buttons or placeholder text.
- Seed data ensures reviewer can demo immediately.
- Commit history is readable and shows staged engineering work.